

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To understand the constraint satisfaction problem in problem formulation	
Experiment No: 13	Date:19/03/2026	Enrolment No:92301733054

Aim: To understand the constraint satisfaction problem in problem formulation

IDE: Google Colab

Theory:

The Constraint Satisfaction Problem (CSP) is a fundamental concept in computer science and artificial intelligence, playing a crucial role in solving a wide range of real-world problems. At its core, CSP involves finding a solution to a problem where the solution must satisfy a set of constraints that define the problem's requirements and limitations. These constraints restrict the possible values that variables can take, leading to a search for a combination of values that satisfies all constraints simultaneously.

CSPs can be formulated in various domains, including scheduling, planning, resource allocation, configuration, and design. Examples of CSP applications include job scheduling, timetabling, Sudoku puzzles, map coloring, and the traveling salesman problem. In each case, the problem consists of a set of variables, each with a domain of possible values, and a set of constraints that specify allowable combinations of variable values.

The basic components of a CSP include:

1. **Variables:** These represent the entities whose values need to be determined. Each variable has a domain, which is a set of possible values it can take.
2. **Domains:** A domain is the set of values that a variable can take. Domains can be finite or infinite, discrete or continuous, depending on the problem domain.
3. **Constraints:** Constraints define the relationships among variables and restrict the combinations of values they can take. Constraints can be unary (applying to a single variable), binary (between two variables), or higher-order (involving three or more variables).

The goal in solving a CSP is to find an assignment of values to variables such that all constraints are satisfied. This involves searching through the space of possible assignments to find a solution or determine that no solution exists. The search process typically involves backtracking, constraint propagation, and variable ordering heuristics to efficiently explore the search space.

One common approach to solving CSPs is backtracking search, which systematically explores the space of possible assignments by making choices and backtracking when a dead-end is reached. Backtracking search is guided by variable and value ordering heuristics to prioritize the search for promising solutions and avoid exploring unpromising regions of the search space.

Another technique used in CSP solving is constraint propagation, which involves enforcing constraints locally to reduce the domain of variables and prune the search space. Constraint propagation techniques include arc consistency, domain reduction, and constraint propagation algorithms such as AC-3 and AC-4.

In addition to backtracking search and constraint propagation, other methods for solving CSPs include local search algorithms, genetic algorithms, and constraint satisfaction neural networks. Each approach has its strengths and weaknesses, making them suitable for different types of CSPs and problem domains.



Marwadi University
Faculty of Technology
Department of Information and Communication Technology

Subject: Artificial Intelligence (01CT0616)

Aim: To understand the constraint satisfaction problem in problem formulation

Experiment No: 13

Date:19/03/2026

Enrolment No:92301733054

Despite the challenges involved in solving CSPs, they offer a powerful framework for modeling and solving complex combinatorial optimization problems. By formulating real-world problems as CSPs and applying appropriate solving techniques, researchers and practitioners can tackle a wide range of practical problems efficiently and effectively.

Methodology:

1. Mention the constraints for sudoku solution
2. Solve the sudoku using the CSP approach

Program (Code):

```
# Function to check if it is safe to place num at mat[row][col]
def isSafe(mat, row, col, num):
    # Check if num exists in the row
    for x in range(9):
        if mat[row][x] == num:
            return False
    # Check if num exists in the col
    for x in range(9):
        if mat[x][col] == num:
            return False
    # Check if num exists in the 3x3 sub-matrix
    startRow = row - (row % 3)
    startCol = col - (col % 3)

    for i in range(3):
        for j in range(3):
            if mat[i + startRow][j + startCol] == num:
                return False
    return True

# Function to solve the Sudoku problem
def solveSudokuRec(mat, row, col):
    # base case: Reached nth column of the last row
    if row == 8 and col == 9:
        return True
    # If last column of the row go to the next row
    if col == 9:
        row += 1
        col = 0

    # If cell is already occupied then move forward
    if mat[row][col] != 0:
        return solveSudokuRec(mat, row, col + 1)

    for num in range(1, 10):
        # If it is safe to place num at current position
        if isSafe(mat, row, col, num):
```



Marwadi University
Faculty of Technology
Department of Information and Communication Technology

Subject: Artificial Intelligence (01CT0616)

Aim: To understand the constraint satisfaction problem in problem formulation

Experiment No: 13

Date:19/03/2026

Enrolment No:92301733054

```
mat[row][col] = num
if solveSudokuRec(mat, row, col + 1):
    return True
mat[row][col] = 0
```

```
return False
```

```
def solveSudoku(mat):
    solveSudokuRec(mat, 0, 0)
```

```
if __name__ == "__main__":
    mat = [
        [3, 0, 6, 5, 0, 8, 4, 0, 0],
        [5, 2, 0, 0, 0, 0, 0, 0, 0],
        [0, 8, 7, 0, 0, 0, 0, 3, 1],
        [0, 0, 3, 0, 1, 0, 0, 8, 0],
        [9, 0, 0, 8, 6, 3, 0, 0, 5],
        [0, 5, 0, 0, 9, 0, 6, 0, 0],
        [1, 3, 0, 0, 0, 0, 2, 5, 0],
        [0, 0, 0, 0, 0, 0, 0, 7, 4],
        [0, 0, 5, 2, 0, 6, 3, 0, 0]
    ]
```

```
solveSudoku(mat)
```

```
for row in mat:
    print(" ".join(map(str, row)))
```

Results:

```
solveSudoku(mat)

for row in mat:
    print(" ".join(map(str, row)))

... 3 0 6 5 0 8 4 0 0
     5 2 0 0 0 0 0 0 0
     4 8 7 0 0 0 0 3 1
     0 0 3 0 1 0 0 8 0
     9 0 0 8 6 3 0 0 5
     0 5 0 0 9 0 6 0 0
     1 3 0 0 0 0 2 5 0
     0 0 0 0 0 0 0 7 4
     0 0 5 2 0 6 3 0 0
```

Observation and Result Analysis: I have observed that the Constraint Satisfaction Problem (CSP) approach efficiently solves problems like Sudoku and N-Queens by systematically exploring possible solutions using backtracking. The constraints help in reducing unnecessary computations by pruning invalid possibilities early. The program successfully generates correct solutions, demonstrating the effectiveness of CSP in handling combinatorial problems. Overall, the implementation highlights how constraints and recursion work together to achieve optimal results.



Marwadi University
Faculty of Technology
Department of Information and Communication Technology

**Subject: Artificial
Intelligence (01CT0616)**

**Aim: To understand the constraint satisfaction problem in problem
formulation**

Experiment No: 13

Date:19/03/2026

Enrolment No:92301733054

Post Lab Exercise:

Write the code and output for N-Queens problem. Take the value of N from the user.

Program Code:

```
def isSafe(mat, row, col):
    n = len(mat)

    # Check this col on upper side
    for i in range(row):
        if mat[i][col]:
            return 0

    # Check upper diagonal on left side
    i, j = row - 1, col - 1
    while i >= 0 and j >= 0:
        if mat[i][j]:
            return 0
        i -= 1
        j -= 1

    # Check upper diagonal on right side
    i, j = row - 1, col + 1
    while i >= 0 and j < n:
        if mat[i][j]:
            return 0
        i -= 1
        j += 1

    return 1

# Recursive function to place queens
def placeQueens(row, mat, result):
    n = len(mat)

    # base case: If all queens are placed
    if row == n:

        # store current solution
        ans = []
        for i in range(n):
            for j in range(n):
                if mat[i][j]:
                    ans.append(j + 1)
            result.append(ans)
        return
```



Marwadi University
Faculty of Technology
Department of Information and Communication Technology

Subject: Artificial Intelligence (01CT0616)

Aim: To understand the constraint satisfaction problem in problem formulation

Experiment No: 13

Date:19/03/2026

Enrolment No:92301733054

```
# Consider the row and try placing
# queen in all columns one by one
for i in range(n):
```

```
    # Check if the queen can be placed
    if isSafe(mat, row, i):
        mat[row][i] = 1
        placeQueens(row + 1, mat, result)
```

```
    # backtrack
    mat[row][i] = 0
```

```
# Function to find all solutions
def nQueen(n):
```

```
    # Initialize the board
    mat = [[0] * n for _ in range(n)]
    result = []
```

```
    # Place queens
    placeQueens(0, mat, result)
```

```
    return result
```

```
if __name__ == "__main__":
    n = 4
    result = nQueen(n)
    for ans in result:
        print(" ".join(map(str, ans)))
```

Result:

```
✓ ... 2 4 1 3
    3 1 4 2
```